

LoanFit AI - Web Platform PRD & TRD

Concept: A borrower-first **web platform** that finds the loan most suitable for an individual using total borrowing cost, affordability, eligibility, lender terms and user preferences—not just the lowest advertised interest rate.

Updated specifically for a desktop/responsive web architecture.

1. PRODUCT REQUIREMENTS DOCUMENT (PRD)

Phase 1 - Foundation & Borrower Profile

- **Web Authentication:** User account creation utilizing standard web cookies/sessions via Supabase.
- **Requirement Selection:** Loan type and requirement selection.
- **Data Entry (Desktop Optimized):** Multi-column web forms for capturing income, expenses, existing EMI, CIBIL score, employment, and location efficiently.
- **Preferences:** Sliders/toggles for lowest cost, EMI, eligibility, fees, flexibility.
- **Outcome:** Working borrower profile and loan requirement web flow.

Phase 2 - Loan Intelligence & Comparison

- **Structured Database:** Lender/loan-product database (Interest rate, fees, tenure, eligibility).
- **Calculations:** EMI, total interest, repayment and known-charge calculation.
- **Web UI Comparison:** Comprehensive desktop-optimized data tables for side-by-side comparison with source and last-verified date.
- **Outcome:** Accurate loan comparison and cost engine viewable on wide screens.

Phase 3 - Personalized Recommendation

- **Filtering:** Hard eligibility filtering and affordability/obligation analysis.
- **Scoring:** Net Cost of Borrowing calculation and MCDA/MAUT suitability scoring.
- **Simulation:** Personalized ranking and interactive what-if web simulator.
- **Outcome:** Top personalized loans with a transparent suitability score.

Phase 4 - AI, XAI & Integrations

- **Explainable Results:** 'Why this loan?' natural language outputs based on SHAP math.
- **Web UI Chatbot:** Grounded AI loan advisor integrated as a persistent sidebar next to recommendation tables.
- **Architecture:** RAG using verified product information & Bhashini-ready multilingual layer.
- **Future Pipeline:** Account Aggregator integration with web-based consent flows.
- **Outcome:** Explainable, multilingual financial assistant running directly in the browser.

Phase 5 - Production, Security & Scale

- **Security:** Supabase RLS, secure web authentication (HTTPS/WSS) and audit logs.
- **Compliance:** Consent and privacy controls optimized for web data collection.
- **DevOps:** Monitoring, model/version tracking and data freshness. Real lender/partner API integrations.
- **Outcome:** Production-ready, scalable web fintech architecture.

2. TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Stack: Next.js + React + TypeScript + Tailwind CSS (Responsive Web); Supabase Auth + PostgreSQL + Edge Functions; Python/FastAPI for ML when needed; scikit-learn/SHAP; Vercel (Web Hosting) + Supabase + Render; GitHub Actions.

Web Architecture Flow:

User Browser → Next.js Web App → Supabase Auth/API → PostgreSQL → Eligibility Cost Engine → MCDA/MAUT → Ranking XAI - AI Advisor.

External adapters: verified lender data, authorized Account Aggregator ecosystem, Bhashini and optional DigiLocker.

Core Database Schema

Table	Key fields	Purpose
borrower_profiles	id, user_id, income, expenses, existing_emi, credit_score	Financial profile
loan_requirements	user_id, loan_type, amount, tenure, max_emi	Borrowing request
lenders	id, name, type, status	Lender registry
loan_products	lender_id, type, rates, fees, tenure, eligibility, source, last_verified_at	Loan catalog
recommendation_runs	user_id, model_version, created_at	Recommendation execution
recommendation_results	run_id, product_id, EMI, total_cost, utility_score, rank	Personalized results
consent_records	user_id, provider, purpose, status, timestamps	Consent tracking
audit_logs	user_id, action, resource, created_at	Audit trail

API & Integration Requirements

- **POST /api/v1/loan-profile:** create/update borrower requirements.
- **POST /api/v1/recommendations:** personalized ranking.
- **POST /api/v1/calculator/emi:** deterministic EMI calculation.
- **POST /api/v1/simulations:** what-if scenarios.
- **POST /api/v1/assistant:** grounded AI advisor.
- **External:** authorized AA ecosystem, Bhashini, optional DigiLocker and lender/partner APIs.

Security & RLS (Web Focus)

Enable Supabase Row Level Security on user-specific tables. Users access only their own profiles, requirements, consents and results. Keep lender-product data read-only for normal users. Use HTTPS, secure browser sessions (HttpOnly cookies preferred), server-side secrets, validation, rate limiting, audit logs and data minimization.

5-Phase Technical Roadmap

Phase	Build	Technology
1	Web Auth + profile forms + requirements	Next.js, Supabase

Phase	Build	Technology
2	Loan catalog tables + EMI + side-by-side comparison	PostgreSQL, Edge Functions
3	Eligibility + affordability + Net Cost + MCDA	TypeScript/Python
4	XAI + persistent AI sidebar + RAG + multilingual/AA adapters	LLM API, SHAP, Bhashini
5	RLS + privacy + monitoring + CI/CD + Vercel Scale	Supabase, Vercel, Render, GitHub Actions

36-Hour Hackathon Priority

Build Phase 1, Phase 2, and Phase 3 first. Use clearly labeled demo lender data. Add a lightweight Phase 4 AI explanation via a web sidebar after the deterministic recommendation engine works. Treat AA, DigiLocker, UPI and production approval-probability ML as future work.

Research Positioning

Do not claim that AI, MCDA, XAI, RAG, loan comparison or Account Aggregator are individually novel. The potential contribution is their borrower-centric web integration: structured/permissioned financial inputs + complete borrowing-cost modeling + affordability/eligibility constraints + MCDA/MAUT ranking + faithful explanations.